9.3 stringr: Summary

A catalog. Open it when you forget which str_ function to call.

Where you are. This is a catalog, not the next lesson. Open it when you forget which `str_` function to call. The lessons continue with note 9.5.

`stringr` (part of the tidyverse) gives consistent, readable functions for string work. The main functions start with `str_`, take the **string first** and the **pattern second**, and are vectorized. Patterns are regex (via the `stringi`/ICU engine) unless wrapped in a helper.

```
library(stringr)
```

1. Detect and count

Function	What it does	Example
`str_detect(x, p)`	TRUE/FALSE: does the pattern occur?	`str_detect("apple", "pp")` → `TRUE`
`str_count(x, p)`	Number of matches	`str_count("banana", "a")` → `3`
`str_starts(x, p)`	Starts with pattern?	`str_starts("apple", "a")` → `TRUE`
`str_ends(x, p)`	Ends with pattern?	`str_ends("apple", "e")` → `TRUE`
`str_which(x, p)`	Indices of elements that match	`str_which(c("a","b","ab"), "a")` → `1 3`
`str_equal(x, y)`	Compare strings (Unicode-aware)	

2. Extract and locate

Function	What it does	Example
`str_extract(x, p)`	First match in each string	`str_extract("ab12cd34", "\\d+")` → `"12"`
`str_extract_all(x, p)`	All matches (returns a list)	→ `"12" "34"`
`str_match(x, p)`	First match **plus capture groups** (matrix)	`str_match("2026-10", "(\\d+)-(\\d+)")`
`str_match_all(x, p)`	Same, for all matches	
`str_locate(x, p)`	Start and end position of first match	
`str_locate_all(x, p)`	Positions of all matches	
`str_subset(x, p)`	Return only the elements that match	`str_subset(c("cat","dog"), "c")` → `"cat"`

3. Replace and remove

Function	What it does	Example
`str_replace(x, p, r)`	Replace first match	`str_replace("aaa", "a", "b")` → `"baa"`
`str_replace_all(x, p, r)`	Replace all matches	→ `"bbb"`
`str_remove(x, p)`	Delete first match	
`str_remove_all(x, p)`	Delete all matches	
`str_replace_na(x, r)`	Replace `NA` with a string	

Backreferences in replacements use `\\1`: `str_replace("John Smith", "(\\w+) (\\w+)", "\\2, \\1")` → `"Smith, John"`. You can also pass a named vector for several replacements at once: `str_replace_all(x, c("a" = "1", "b" = "2"))`.

4. Length and subsetting

Function	What it does	Example
`str_length(x)`	Number of characters	`str_length("hello")` → `5`
`str_sub(x, start, end)`	Extract by position (negatives count from the end). One character is `str_sub(x, i, i)`	`str_sub("hello", 2, 4)` → `"ell"`
`str_sub(x, 1, 2) <- "XX"`	Replace a substring by position	
`str_trunc(x, width)`	Shorten with `...`	`str_trunc("abcdefgh", 6)` → `"abc..."`

5. Case

Function	What it does
`str_to_lower(x)`	lowercase
`str_to_upper(x)`	UPPERCASE
`str_to_title(x)`	Title Case
`str_to_sentence(x)`	Sentence case

6. Whitespace and padding

Function	What it does	Example
`str_trim(x, side)`	Remove leading/trailing whitespace (`"both"`, `"left"`, `"right"`)	`str_trim(" hi ")` → `"hi"`
`str_squish(x)`	Trim **and** collapse inner repeated spaces	`str_squish(" a b ")` → `"a b"`
`str_pad(x, width, side, pad)`	Pad to a fixed width. The default side is `"left"`	`str_pad("7", 3, pad = "0")` → `"007"`
`str_wrap(x, width)`	Wrap text into lines of a set width	
`str_trunc()`	See above	

7. Combine, split, repeat

Function	What it does	Example
`str_c(..., sep, collapse)`	Concatenate (like `paste`, but `NA`-propagating)	`str_c("a", "b", sep = "-")` → `"a-b"`
`str_flatten(x, collapse)`	Collapse a vector into one string	`str_flatten(c("a","b"), ", ")` → `"a, b"`
`str_flatten_comma(x, last)`	Comma list. `last = ", and "` inserts "and" before the final item, and skips the extra comma when there are only two	`str_flatten_comma(c("a","b","c"), ", and ")` → `"a, b, and c"`
`str_split(x, p)`	Split into pieces (returns a list)	`str_split("a,b,c", ",")`
`str_split_fixed(x, p, n)`	Split into exactly `n` columns (matrix)	
`str_split_1(x, p)`	Split a single string into a vector	
`str_dup(x, times)`	Repeat a string	`str_dup("ab", 3)` → `"ababab"`

8. Interpolation and formatting

Function	What it does	Example
`str_glue("...{var}...")`	Insert R values into text	`str_glue("Hi {name}")`
`str_glue_data(.x, ...)`	Same, using columns of a data frame	
`sprintf()`	C-style formatting (base R)	
`str_like(x, p)`	SQL-style `LIKE` matching (`%` and `_`)	

9. Sorting, order, viewing, encoding

Function	What it does
`str_sort(x)`	Sort strings (locale-aware)
`str_order(x)`	Return sort order indices
`str_rank(x)`	Rank strings
`str_view(x, p)`	Visually highlight matches (great for debugging regex)
`str_conv(x, enc)`	Convert encoding
`str_escape(x)`	Escape regex special characters
`str_unique(x)`	Unique strings

10. Pattern helpers (modifiers)

Wrap the pattern to change how it is interpreted:

Helper	Meaning
`regex(p, ignore_case = TRUE, multiline = TRUE)`	Regex with options
`fixed(p)`	Literal match, no regex (faster, safe for special characters)
`coll(p)`	Locale-aware comparison
`boundary("word")`	Split or match on word, sentence, or character boundaries

```
str_detect("A.B", fixed("."))           # literal dot
str_detect("Apple", regex("apple", ignore_case = TRUE))
str_split("Hello big world", boundary("word"))
```

11. Handy built-in data

- `fruit`, `words`, `sentences`: sample vectors for practice

12. Common patterns in practice

```
# Clean messy text
x |> str_squish() |> str_to_lower()

# Filter a data frame
df |> filter(str_detect(name, "^A"))

# Extract numbers
str_extract_all(x, "\\d+")

# Pull out groups
str_match(x, "(\\w+)@(\\w+)\\.com")

# Zero-pad IDs
str_pad(id, 5, pad = "0")
```

Key points to remember

1. **Input first, pattern second**: this makes `stringr` pipe-friendly (`|>`).
2. **`_all` suffix** = every match; no suffix = first match only.
3. **Type `"\\d"` for a digit and `"\\w"` for a word character.**
4. **Output type hints**: `str_detect` returns a logical vector, `str_extract` returns a character vector, `str_extract_all` and `str_split` return lists.
5. **`NA` handling**: most functions return `NA` for `NA` input.
6. **Use `fixed()`** when you want a plain-text match with no regex.

stringr vs. base R equivalents

stringr	Base R
`str_detect`	`grepl`
`str_replace_all`	`gsub`
`str_extract`	`regmatches` + `regexpr`
`str_split`	`strsplit`
`str_sub`	`substr`
`str_length`	`nchar`
`str_c`	`paste0`
`str_to_upper`	`toupper`